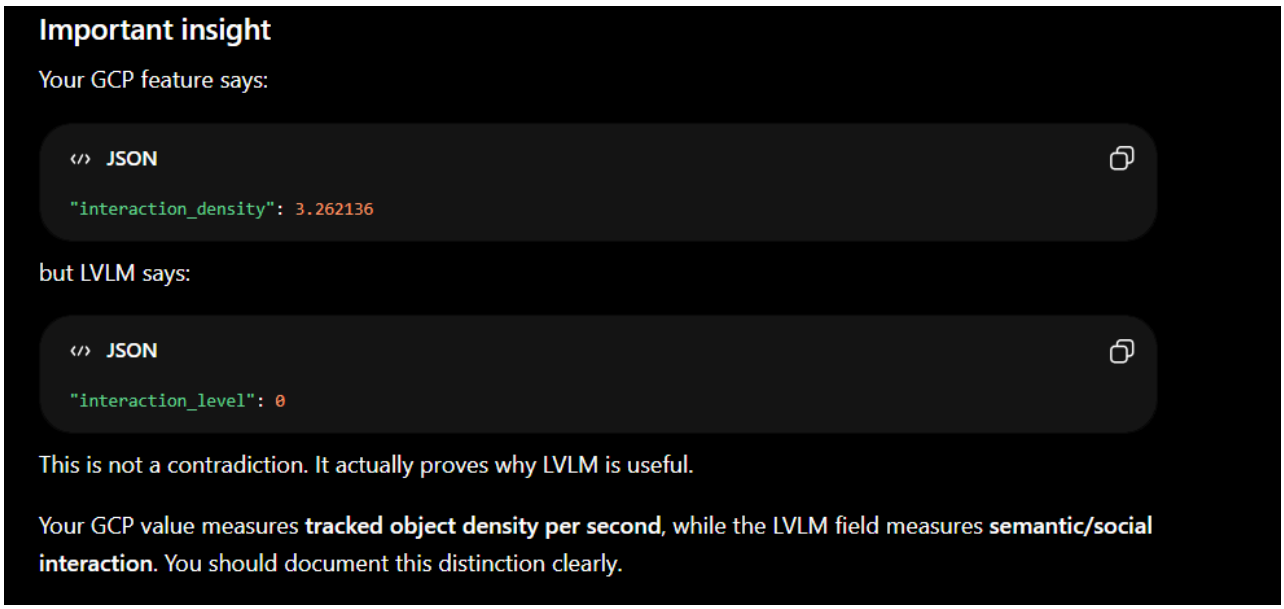
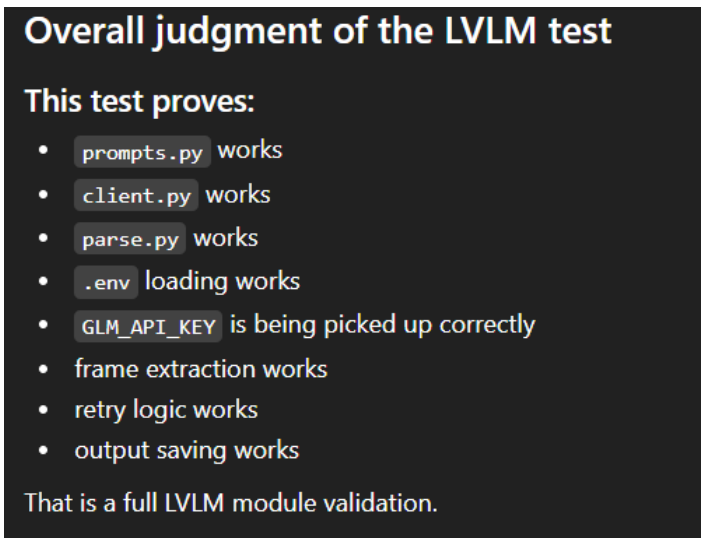
Findings:

- LVLM & LLM cost (API's) (LVLM - getting a slice out of the weekly quota of my friend's 720$ yearly subscription - need to research how much is it for the GLM key specifically) V
- (token limitations - check with Lezef) translate token amounts to real life currency (28M tokens for one test usage on 1 video multiple times across a week, 14M on the last 24hrs - didnt even scratched the 1% token limitation) - V (28M is including KV cache - stores your computed information and only computes new one, which means in practice I spent about 8M tokens - 70% less, in the last 24hrs I spent 14M*0.3 = 4M tokens)
-



Which proves here why having GCP layer and LVLM layer is very effective and helpful. (each gives their respective point of view and data knowledge)

-



- https://console.cloud.google.com/billing/01413C-8B03CB-CC22D4/reports;timeRange=CUSTOM_RANGE;from=2025-05-01;to=2026-05-31?hl=en&project=green-calling-481021-r2
  Findings about billing in the period used in GCP (1 year overview - as of May 2026)

- a note about video id: ACCEDE09231: it says in the summary: LVLM correctly identifies the car interior/residential driving scene. It supports the algorithmic Calm phase: low shot frequency, low interaction density, and no direct interaction between the two people.
  - but, it has a communication between the 2 visible characters, although one sided, interaction is still there being the woman talks, and the man glances at her momentarily. (might need to further tune this)
- Ran into unexpected bug when running the script consecutively on 2 videos one after the other, the first video succeeded, but the second video execution failed due to:
  "The service may be temporarily overloaded, please try again later"
  After analyzing:
  - GCP stage succeeded
  - VideoFeatures.json was saved
  - LVLM stage started
  - frame extraction succeeded
  - LVLM summary request failed due to API overload
  - pipeline stopped before VideoInterpretation.json was completed

  (added lvlm fallbacks for error with null replacement while GLM is overloaded)
  - After adding error fallbacks to the script and rerunning the script on the same failed video it succeeded, with no overload issues.
- During week 4 programming and implementation, I saw in the LVLM script, the usage of extracting frames being performed twice unnecessarily, so I fixed it to be only once, and used the same frames however much necessary.
- Also found that there were many different functions being used across scripts in the pipeline with them being defined each time multiple times unnecessarily, so I prompted to create only one function to be used across scripts, (not circular), on whichever script was the oldest, and to the other scripts using that same function, with a designated import.
- Additional findings regarding week 3 and week 4 result comparison is in git Final Project Repository in Final-Project/evaluation respectively.
- Week 4 proved that LVLM can produce structured semantic metadata, but the interaction-level field was too conservative.
- In Week 5, we tuned the structured prompt and evaluated expected vs predicted interaction levels while also collecting numeric feature distributions for threshold/range tuning.
- Timing check for GCP layer:
  Video id (Video length):
  09237(11 secs) - 28.58 s, 09238 - 27.18 s, 09239 - 18.51 s,
  09240 (8.16 secs) - 18.46 s, 09241 (8.75 secs) - 22.94 s, 09242 (9.75 secs) - 25.27 s,
  09244 (8.29 secs) - 32.51 s, 09245 (10.41 secs) - 22.41 s, 09246 (8.41 secs) - 19 s,
  09247 (10.37 secs) - 31.76s, 09248 (10.29 secs) - 29.53 s, 09249 (11.04 secs)- 21.78 s,
  09250 (11.62 secs) - 20.88 s.
- | ACCEDE09244 | retry needed | GCS connection reset during blob.exists |
- current normalization ranges are too wide for shot_frequency and interaction_density, while object_entropy and human_presence_ratio are mostly reasonable.

- interaction_density from GCP and lvlm_interaction_level do not measure the exact same thing: GCP interaction_density is based on tracked object density per second.
  LVLM interaction_level is a semantic estimate of how much the visible entities interact.
  Object entropy - מדד לאי סדר של מערכת חומרית המורכבת מחלקיקים רבים (מדד לאי סדר של חפצים, אובייקטים)
- All 21 videos had "Calm" phase classification, before updating config.py
  GCP findings after testing on 20 videos:

Summary of Main Findings

feature	finding	recommended action
`shot_frequency`	Current max `2.0` is too wide	Consider lowering to `1.0`
`object_entropy`	Range `0–4` is acceptable, but Static entropy threshold may be too strict	Keep range for now; revisit threshold later
`interaction_density`	Current max `10.0` is too wide	Lower to `5.0`
`human_presence_ratio`	Naturally bounded and working as expected	Keep `0–1`

- Mode 1 — full
  local video → GCS → GCP analysis → VideoFeatures.json → VideoInterpretation.json
  Mode 2 — gcp_tuning
  existing VideoFeatures.json → recompute VideoInterpretation.json only
  Mode 3 — lvlm_tuning
  existing VideoFeatures.json + local video → recompute VideoInterpretation.json with LVLM fields

Why modes are better

Because each mode has a clear purpose:

mode	uses GCS/GCP API?	uses LVLM API?	purpose
`full`	Yes	No by default	New videos
`gcp_tuning`	No	No	Fast numeric retuning
`lvlm_tuning`	No	Yes	Prompt/semantic retuning

- After 1st tuning: (compared to before - 21 Calm videos)
  Phase Distribution
phase	count
Calm	18
Dynamic	0
Dense	3
Static	0
Unknown	0

- Changing `config.py` changes:
  features_norm, scene_complexity_score, narrative_phase
  (makes sense that features raw remain unchanged.)
- Implemented phase thresholds changes - v2 - for further testing.
- **## Observations**
  - After updating the normalization ranges, the raw feature statistics stayed unchanged, which is expected because raw features are produced from GCP metadata and do not depend on `config.py`.
  - The phase distribution changed from all Calm to 18 Calm and 3 Dense, showing that the updated normalization ranges made Dense reachable.
  - Dynamic is still never predicted because the current dynamic shot-frequency threshold is 0.65, while the maximum observed normalized shot frequency is only 0.4660.
  - Static is still never predicted because the current static entropy threshold is 0.35, while the minimum observed normalized object entropy is approximately 0.3805.
  - Further tuning should focus on phase thresholds, especially
  `dynamic_shot_frequency_min` and `static_entropy_max`.
- **## Suggested Next Tuning Step (Suggested Range Updates)**
component	current_value	suggested_value	reason
dense_entropy_min	0.65	0.70	Object entropy median is already high, so Dense should require stronger entropy.
dynamic_shot_frequency_min	0.65	0.35	Current value is unreachable because max observed normalized shot frequency is 0.4660.
dynamic_density_max	0.65	0.75	Allows moderately busy fast-cut scenes to still be Dynamic.
static_shot_frequency_max	0.35	0.20	Static should require very slow cutting.
static_density_max	0.35	0.30	Static should require low tracked-object density.
static_entropy_max	0.35	0.50	Current value is too strict because even the lowest observed entropy is above 0.35.
- After 2nd tuning:
  - after v1 changes (config.py - NormalizationRanges) we got this distribution: from 21 videox - Calm to: 18 Calm, 3 Dense,
  - after v2 changes (config.py - PhaseThresholds) we got the following: Calm - 15, Dynamic - 1, Dense - 2, Static - 3.
- Lets be reminded of these definitions:
  - Dense if Scene is crowded + complex.
  - Dynamic if Scene is fast-paced but not crowded.
  - Static if Scene is very calm / minimal.
  - else Calm - everything else - Balanced / normal scene.
- Initial / v0: 21 Calm
  After v1 normalization ranges: 18 Calm, 3 Dense
  After v2 phase thresholds: 15 Calm, 1 Dynamic, 2 Dense, 3 Static

Week 6 for LVLM interaction lvl tuning:

Before tuning Structured prompts: - V1

```
Interaction level scale:
0 = no visible interaction between entities.
    Examples: people are present but doing separate activities; one person walks alone.
1 = weak or indirect interaction.
    Examples: one person speaks while another listens; one person glances at another; people share the same activity but do not clearly respond to each other.
2 = clear mutual interaction.
    Examples: two people talk to each other, react to each other, gesture toward each other, or coordinate actions.
3 = strong or intense interaction.
    Examples: argument, physical struggle, urgent emotional exchange, chase, direct confrontation, intense cooperation.

Rules:
- interaction_level must be based on visible behavior only.
- If there is speaking, looking, gesturing, or reaction between people, do not return 0.
- Return 0 only when entities are present but no direct or indirect interaction is visible.
- Include a short interaction_evidence string explaining the chosen level.
- Do not include markdown.
- Do not include text outside the JSON.
```

After tuning: - V2

Interaction level scale:

0 = No visible interaction between entities
Use only when one person is alone, or multiple people are present but clearly focused on separate unrelated activities.

1 = Weak or indirect interaction.
Use when people share a context or activity but interaction is limited.

Examples: one person speaks while another listens, brief glances, shared car ride, one-sided communication, passive observation.tion.

2 = clear mutual interaction.
Use when people visibly communicate, respond to each other, gesture toward each other, coordinate actions, or appear engaged in the same exchange.

3 = Strong / intense interaction
Use for argument, confrontation, chase, physical struggle, urgent cooperation, or intense emotional exchange.

Important rules:

- Do not return 0 if there is visible speaking, listening, glancing, reacting, gesturing, or shared coordinated activity.
- If people are in a car together and one appears to speak or react to the other, choose 1 or 2 depending on clarity.
- Use 0 only when there is no visible social or task-based interaction.
- Always include interaction_evidence explaining the chosen level.

Manual Scoring for V2 Tuning LVLM:

Use this scoring:

Exact match = Yes
Off by 1 but evidence is reasonable = Partial
Wrong direction or clearly unsupported = No

- While running batch videos on LVLM - it is a recommended practice to use **small batches**, small amounts - so the user wont wait long and so it wont overbear the LVLM model.
  (I ran a batch of 4 videos and it took a few mins)

- Need to edit collect_feature_stats.py to include handling the lvlm changes on a certain number of videos tested, and write down their findings onto week 6 evaluation md file. (make it more general for both cases - perhaps change name) - maybe make a new one for LVLM.

  V2 - after tuning v1:
  LVLM interaction rows collected:

  - ACCEDE09230: expected=0 | predicted=0 | match=Yes
  - ACCEDE09231: expected=1 | predicted=None | match=No
  - ACCEDE09232: expected=0 | predicted=0 | match=Yes
  - ACCEDE09233: expected=2 | predicted=None | match=No

    - The reason it was predicted NONE or N/A was because the LVLM have had this error:
      `BadRequestError: Error code: 400 - {'contentFilter': [{'level': 2, 'role': 'user'}], 'error': {'code': '1301', 'message': 'System detected potentially unsafe or sensitive content in input or generation. Please avoid using prompts that may generate sensitive content. Thank you for your cooperation.'}}`
    - Meaning, the LVLM flagged our definition for lvl 3 interaction_level as sensitive, we need to soften the wording.
      The level 3:
      3 = Strong / intense interaction
      Use for argument, confrontation, chase, physical struggle, urgent cooperation, or intense emotional exchange.
    - After updating:
      3 = Strong or highly active interaction.
      Use when the interaction is intense, urgent, emotionally strong, or involves clear coordinated action between entities.

    - The results update:
      LVLM interaction rows collected:
      - ACCEDE09230: expected=0 | predicted=0 | match=Yes
      - ACCEDE09231: expected=1 | predicted=0 | match=Partial
      - ACCEDE09232: expected=0 | predicted=0 | match=Yes
      - ACCEDE09233: expected=1 | predicted=0 | match=No/Partial

Findings regarding the LVLM tuning:
We went over the interaction evidence for video id:
09231, 09233

Predictions with glm 4.6v flash:
ACCEDE09231: expected 1, predicted 0 → false zero
ACCEDE09233: expected 1, predicted 0 → false zero
(after further investigation and prompt accuracy, I've changed the expected level on 09233 from 2 -> ½ -> 1. (since it does depict a gentle interaction, aswell as 09231).

The LVLM model didnt marked down the interactions because he didnt recognize them and so I tried with the LLM to fix the perception on those cases.

- Now I'm trying a newer LVLM GLM model, instead of "glm-4.6v-flash" were using "glm-5v-turbo".
  The biggest difference apparently with this model is that there is no longer a need for extracting frames from a given video to feed to the LVLM mode, the GLM 5v does it by itself so it saves me from writing alot of the pipeline (which ill not remove and upgrade, only after submitting the final project itself and getting it graded).
- Time tests: 4 first videos (familiar) glm 5v turbo - 5.12 minutes (caffe wifi)
- 4 same videos - glm 4.6v flash -

Predictions with glm 5v turbo:
LVLM interaction rows collected:

- ACCEDE09230: expected=0 | predicted=0 | match=Yes
- ACCEDE09231: expected=1 | predicted=0 | match=Partial
- ACCEDE09232: expected=0 | predicted=0 | match=Yes
- ACCEDE09233: expected=1 | predicted=0 | match=Partial

Prompts:
- v1 - was the original one, v2 is after first changes for the interaction level and important rules with glm 4.6v flash.
- v3 - is after changing interaction level and important rules with glm 5v turbo (after testing once with v2)
With v3 changes - I generalized the structured prompts for interaction level and changed important rules to include conclusions.

Findings so far:
V1: glm-4.6v-flash + original structured prompt
Result: interaction_level was too conservative, many 0s.

V2: glm-4.6v-flash + improved prompt
Result: valid evidence, but still missed subtle interactions.

V3: glm-5v-turbo + same prompt
Goal: test whether stronger model improves interaction_level before adding more prompt constraints.
Result: No major improvement in interaction_level.

V4: glm-5v-turbo + structured prompt v3 (updating interaction level and important rules)
Result:

LVLM interaction rows collected:
- ACCEDE09230: expected=0 | predicted=1 | match=Partial
- ACCEDE09231: expected=1 | predicted=0 | match=Partial
- ACCEDE09232: expected=0 | predicted=0 | match=Yes
- ACCEDE09233: expected=1 | predicted=1 | match=Yes

● Here's the complete list of changes:

Findings after testing twice with the glm 5v: (diagnosis with Claude code)
I ran into unexpected bugs and errors and used Claude code to run over the src directory of the project to dissect potential bugs and errors in the system.
(since I tried with ChatGPT and got lost with the insane amount of solutions it offered and got completely overwhelmed by it)

src/config.py — 1 addition
Added lvlm_enable_thinking: bool = False to ModelSettings, with a comment explaining why it's off (thinking eats the, max_tokens budget and returns empty content).,

src/lvlm/client.py — 4 changes
* call_glm_with_retry() now sends extra_body={"thinking": {"type": "disabled"}} (or "enabled", driven by the new,
* config flag) on every API call — this is the actual bug fix.
* build_result() now raises a clear RuntimeError if the API returns empty content (reporting finish_reason and whether,
reasoning consumed the tokens), instead of letting parsing fail later with the cryptic "No JSON object start" error.
* build_result() now includes "finish_reason" in the returned dict, so it gets saved into lvlm_structured_raw in,
VideoInterpretation.json for future debugging.,

src/pipeline/verify_lvlm_fix.py — new file
Single-video smoke test that runs the same chain as run_lvlm_tuning_mode (extract frames → summary inference → structured inference → parse → validate)

without needing VideoFeatures.json.
Run with python -m src.pipeline.verify_lvlm_fix --video <path>.

Housekeeping (not code): deleted the stale pycache folders so the zip contains only source.

Nothing else was touched — parse.py, prompts.py, validate.py, and all pipeline runners are unchanged.

- After these changes, the code ran successfully with no bugs.

Testing results for 12 videos - the last 8 being unknown to the newer LVLM model:
- one thing to note, I deleted and backed up the 8 new videos's VideoInterpretation.json files.
* took the model 1 minute and 11 seconds to go over the 4 first videos - that he should already be familiar with.
* it took the model 19 seconds to go over the fifth video - meaning the first video that it is not familiar with.
* it took for the model in total for 12 videos - 8 of which are unfamiliar to the specific LVLM model: 3 minutes and 31 seconds - Huge improvement compared to the testing of the first 4 videos which went for 5 minutes and 12 seconds.

LVLM interaction rows collected: 66% accuracy.
- ACCEDE09230: expected=0 | predicted=0 | match=Yes
- ACCEDE09231: expected=1 | predicted=0 | match=Partial
- ACCEDE09232: expected=0 | predicted=0 | match=Yes
- ACCEDE09233: expected=1 | predicted=1 | match=Yes
- ACCEDE09234: expected=2 | predicted=3 | match=Partial
- ACCEDE09235: expected=2 | predicted=2 | match=Yes
- ACCEDE09236: expected=1 | predicted=2 | match=Partial
- ACCEDE09237: expected=3 | predicted=2 | match=Partial
- ACCEDE09238: expected=2 | predicted=2 | match=Yes
- ACCEDE09239: expected=1 | predicted=1 | match=Yes
- ACCEDE09240: expected=1 | predicted=1 | match=Yes
- ACCEDE09241: expected=1 | predicted=1 | match=Yes

- Now testing 20 videos - 12 of which are familiar to the model - and the last 8 are unfamiliar (previous VideoInterpretation.json file got removed and backed up)
  Timings: 3.40 minutes for 12 videos
  5.51 minutes for 21 videos total.

Found 21 videos with expected interaction levels.
Found 20 VideoInterpretation.json files.

LVLM interaction rows collected:
- ACCEDE09230: expected=0 | predicted=1 | match=Partial
- ACCEDE09231: expected=1 | predicted=0 | match=Partial
- ACCEDE09232: expected=0 | predicted=0 | match=Yes
- ACCEDE09233: expected=1 | predicted=1 | match=Yes
- ACCEDE09234: expected=2 | predicted=3 | match=Partial
- ACCEDE09235: expected=2 | predicted=2 | match=Yes
- ACCEDE09236: expected=1 | predicted=2 | match=Partial
- ACCEDE09237: expected=3 | predicted=2 | match=Partial
- ACCEDE09238: expected=2 | predicted=2 | match=Yes
- ACCEDE09239: expected=1 | predicted=1 | match=Yes
- ACCEDE09240: expected=1 | predicted=1 | match=Yes
- ACCEDE09241: expected=1 | predicted=1 | match=Yes
- ACCEDE09242: expected=1 | predicted=0 | match=Partial
- ACCEDE09243: expected=1 | predicted=1 | match=Yes
- ACCEDE09244: expected=2 | predicted=1 | match=Partial
- ACCEDE09245: expected=2 | predicted=1 | match=Partial
- ACCEDE09246: expected=1 | predicted=1 | match=Yes
- ACCEDE09247: expected=0 | predicted=0 | match=Yes
- ACCEDE09248: expected=0 | predicted=0 | match=Yes
- ACCEDE09249: expected=1 | predicted=1 | match=Yes
- ACCEDE09250: expected=2 | predicted=1 | match=Partial

Accuracy score for 20 videos: - Exact match rate: 12/21 = 57.1%
Now testing further for 40 videos.
* before running 40 videos test with the full pipeline: GCP+algorithm+LVLM
I made sure the run_batch_pipeline script is efficient since it had some flaws, then checked a few times while trying to test, if the lvlm fields are properly filled and fixed where was necessary - when it wasnt. Full mode now behaves like this:

No interpretation file → run.
Interpretation exists but LVLM fields are null → run.
Interpretation exists with an LVLM error → run again.
Interpretation exists with valid LVLM results → skip.
`--force` → always run.

* fixed run_full_pipeline script and run_batch_pipeline script from old week3 code to new updated clean code and organized with no duplicate code and functions, to be more efficient.
* it didnt fill out the lvlm fields in the json properly so debugged that and fixed the 2 pipeline scripts accordingly - and then tested 40 videos with timer - then wrote the lvlm updates to a different .md file called After Prompt Update.

40 videos full pipeline took - 28.40 minutes. - 25/40 62% accuracy

LVLM interaction rows collected:
- ACCEDE09230: expected=0 | predicted=1 | match=Partial
- ACCEDE09231: expected=1 | predicted=1 | match=Yes
- ACCEDE09232: expected=0 | predicted=0 | match=Yes
- ACCEDE09233: expected=1 | predicted=1 | match=Yes
- ACCEDE09234: expected=2 | predicted=3 | match=Partial
- ACCEDE09235: expected=2 | predicted=2 | match=Yes
- ACCEDE09236: expected=1 | predicted=2 | match=Partial
- ACCEDE09237: expected=3 | predicted=2 | match=Partial
- ACCEDE09238: expected=2 | predicted=2 | match=Yes
- ACCEDE09239: expected=1 | predicted=1 | match=Yes
- ACCEDE09240: expected=1 | predicted=1 | match=Yes
- ACCEDE09241: expected=1 | predicted=1 | match=Yes
- ACCEDE09242: expected=1 | predicted=1 | match=Yes
- ACCEDE09243: expected=1 | predicted=1 | match=Yes
- ACCEDE09244: expected=2 | predicted=1 | match=Partial
- ACCEDE09245: expected=2 | predicted=1 | match=Partial
- ACCEDE09246: expected=1 | predicted=1 | match=Yes
- ACCEDE09247: expected=0 | predicted=0 | match=Yes
- ACCEDE09248: expected=0 | predicted=0 | match=Yes
- ACCEDE09249: expected=1 | predicted=1 | match=Yes
- ACCEDE09250: expected=2 | predicted=1 | match=Partial
- ACCEDE09251: expected=1 | predicted=0 | match=Partial
- ACCEDE09252: expected=0 | predicted=0 | match=Yes
- ACCEDE09253: expected=0 | predicted=1 | match=Partial
- ACCEDE09254: expected=1 | predicted=0 | match=Partial
- ACCEDE09255: expected=0 | predicted=0 | match=Yes
- ACCEDE09256: expected=0 | predicted=0 | match=Yes
- ACCEDE09257: expected=2 | predicted=2 | match=Yes
- ACCEDE09258: expected=1 | predicted=2 | match=Partial
- ACCEDE09259: expected=2 | predicted=2 | match=Yes
- ACCEDE09260: expected=1 | predicted=0 | match=Partial
- ACCEDE09261: expected=1 | predicted=1 | match=Yes
- ACCEDE09262: expected=2 | predicted=2 | match=Yes
- ACCEDE09263: expected=1 | predicted=0 | match=Partial
- ACCEDE09264: expected=1 | predicted=1 | match=Yes
- ACCEDE09265: expected=1 | predicted=1 | match=Yes
- ACCEDE09266: expected=1 | predicted=1 | match=Yes
- ACCEDE09267: expected=2 | predicted=2 | match=Yes
- ACCEDE09268: expected=1 | predicted=2 | match=Partial
- ACCEDE09269: expected=1 | predicted=2 | match=Partial
- ACCEDE09270: expected=0 | predicted=1 | match=Partial

TODO:

- Learn about what made the change to the prompts - why like this and not in another way.
- Distribute the testing of LVLM across a few days. (V)
- 20 videos~ (1/30 of the database) - check loading time - videos that were never checked. (maybe 40-50) - update if I have trouble (V)
- Use cases, Categorize, attach the documents (work plan, findings)
- Work on a poster (last priority)
- Study the project deeply.
- Check how does the GLM react to unknown videos vs known. (V)
- Isolate the learning of testing a few times on the same videos.
- Check if I can run it in my sleep. - no need (V)
- Check if its possible to improve the run_full_pipeline.py script to not extract frames with the glm 5v turbo - since that model handles this part already. (V)
- Research if the GLM models have reset cache with the API usage or any sort of buffer.

TODO:

- Check how much does the project itself weigh - without videos (if under 100MB) - try with a program like WINRAR.
- Submit the project ZIP after code review today.
- Start editing the Project Book.

README.md:

- project purpose
- architecture
- three pipeline modes
- setup
- environment variables
- GCP credentials
- GLM configuration
- single-video commands
- batch commands
- output schemas
- evaluation workflow
- tests

"pipeline_metadata": {

"schema_version": "1.0",

"algorithm_version": "week6-v1",

"model": "glm-5v-turbo",

```
  "summary_prompt_version": "v1_summary",
  "structured_prompt_version": "v3_interaction"
}
```